

# Eye Snake Localization Using Vision

Student: Chuntung Zhuang,

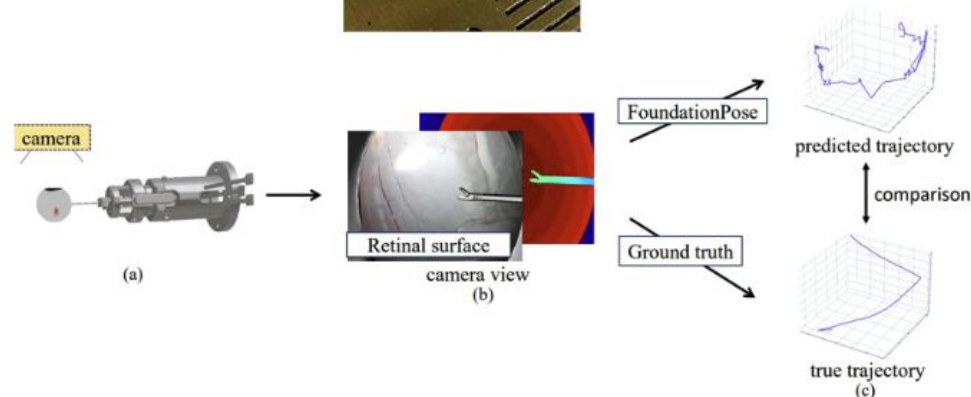
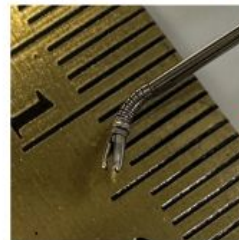
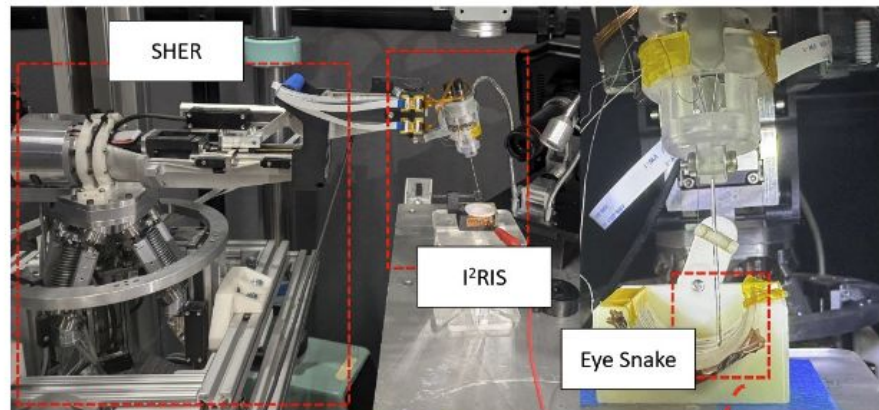
**(Looking for collaborator!!! here [czhuang3@jh.edu](mailto:czhuang3@jh.edu))**

Mentors: Adnan Munawar, Iulian Iordachita, Mojtaba Esfandiari

CIS II - Spring 2026

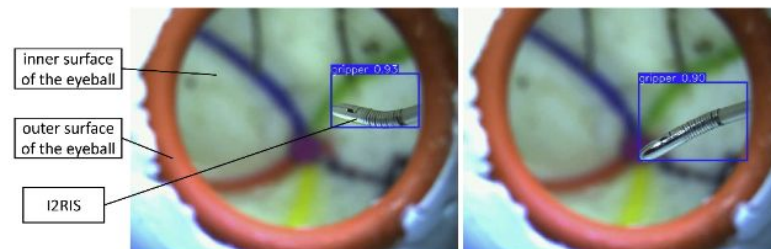
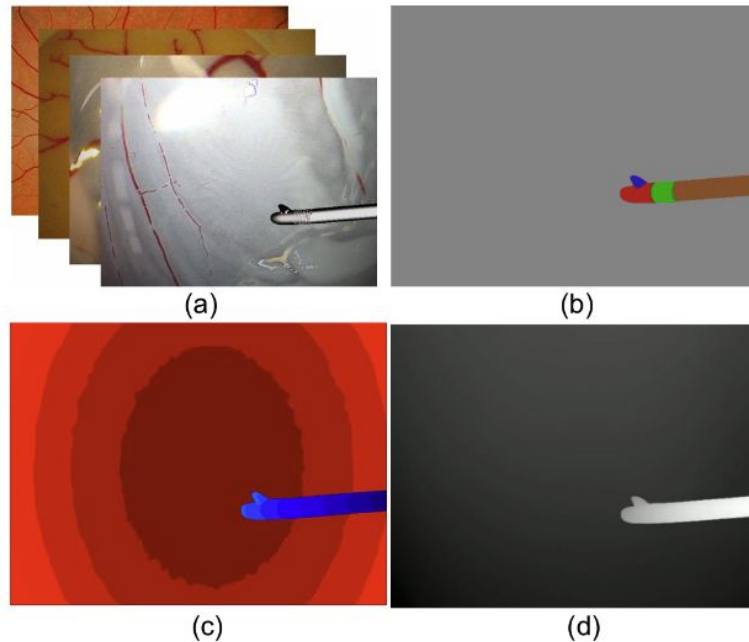
# Goals & Significance: Clinical Need

**Vitreoretinal surgery** → sub-millimeter precision → snake robot!



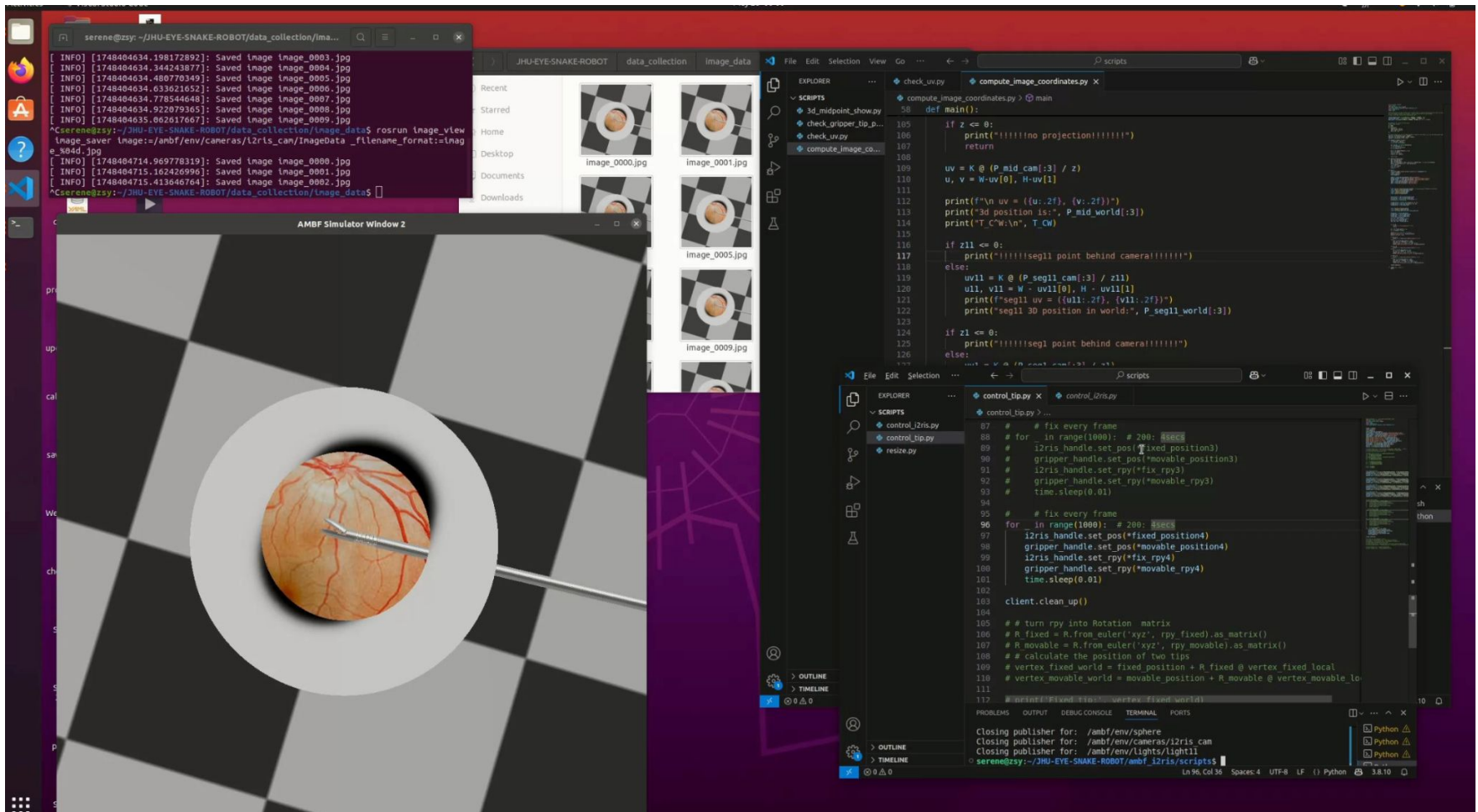
# Goals & Significance: Project Goals

Develop vision-based localization system for the snake robot

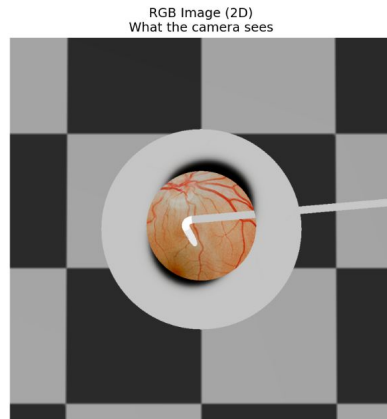


# Technical Approach: Groundwork

- 1) AMBF (Asynchronous Multi-Body Framework)
  - a) - JHU LCSR open-source surgical simulation
- 2) I2RIS snake robot simulation environment (work by previous research)

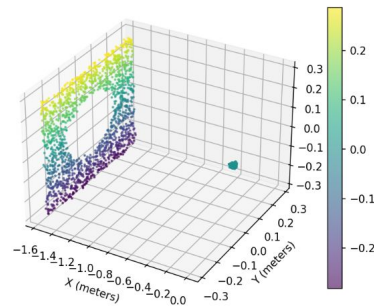


# Technical Approach: AMBF Simulation Data Collection

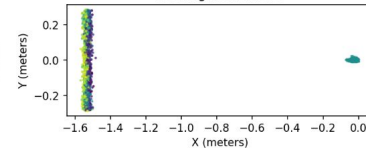


## Understanding Point Clouds

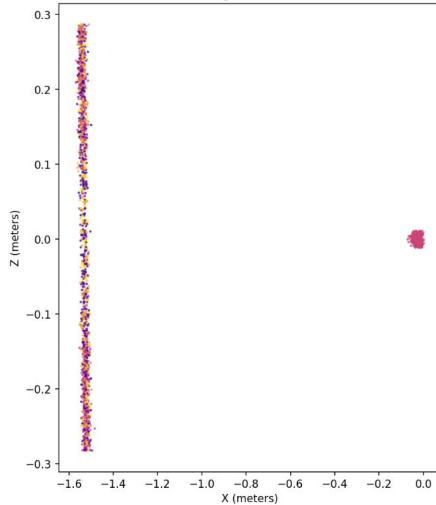
Point Cloud (3D)  
Each dot = a surface point in 3D space



Top-Down View (X-Y)  
Looking from above



Side View (X-Z)  
Looking from the side



### WHAT IS A POINT CLOUD?

A point cloud is a collection of 3D points (X, Y, Z) that represent surfaces in the scene.

### HOW IT'S CREATED:

1. Camera captures RGB image (color)
2. Depth sensor measures distance to each pixel
3. Using camera intrinsics (focal length, etc.), each pixel + depth → 3D point (X, Y, Z)

### COORDINATE SYSTEM:

- X: Left/Right
- Y: Up/Down
- Z: Depth (distance from camera)

### IN THIS DATA:

- 4096 points per sample
- Points capture: eye surface, snake robot, background, etc.
- Used for 3D localization (predict gripper position from point cloud)

### WHY USE POINT CLOUDS?

- Direct 3D information (not just 2D image)
- Scale-invariant (real distances in meters)
- Good for depth-based localization
- PointNet/PointNet++ can process them directly

### POINT CLOUD STATISTICS

Shape: (4096, 3)  
(4096 points × 3 coordinates)

X range: [-1.581, 0.007] m  
Y range: [-0.286, 0.285] m  
Z range: [-0.283, 0.286] m

Mean position:  
X: -1.2011 m  
Y: -0.0001 m  
Z: 0.0018 m

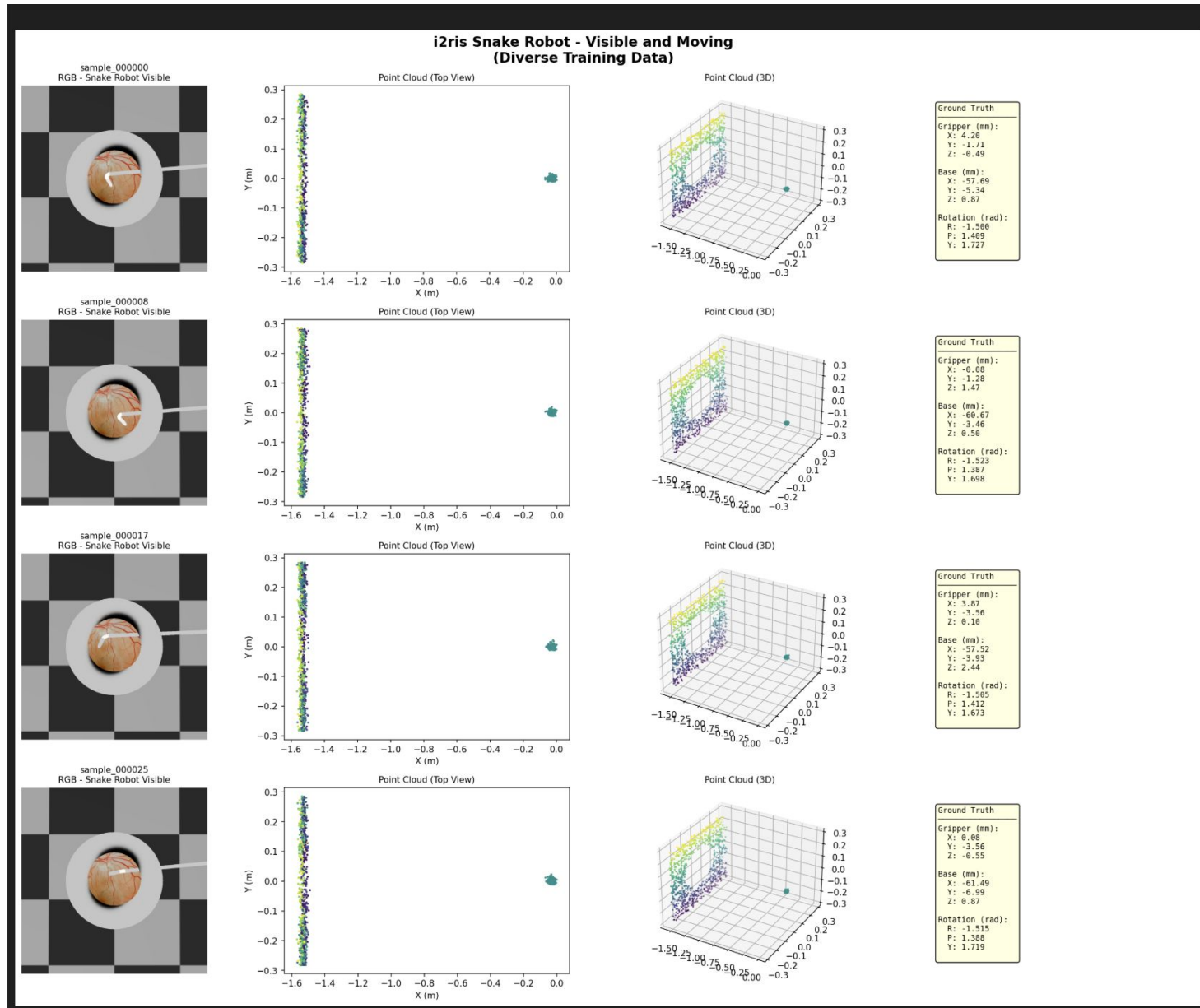
### CAMERA PARAMETERS:

Focal length:  $f_x = f_y = 2935.9$  pixels  
Principal point:  $c_x = c_y = 512$  pixels  
Resolution:  $1024 \times 1024$

### GROUND TRUTH (this sample):

Gripper position:  
X: 4.20 mm  
Y: -1.71 mm  
Z: -0.49 mm

# Technical Approach: AMBF Simulation Data Collection



# Preliminary Results: Approach Comparison

| Approach           | Modality      | Method                       | Error (mm) | Std (mm) | <5mm  | <10mm |
|--------------------|---------------|------------------------------|------------|----------|-------|-------|
| HRNet-Heatmap      | RGB           | HRNet+Softmax                | 2.98       | 1.47     | 93.3% | 100%  |
| Hybrid Fusion      | RGB+PC+Joints | ResNet-18+Point Net+MLP      | 3.49       | 1.23     | 93.3% | 100%  |
| Transformer Fusion | RGB+PC        | ViT+PointNet+Cross-Attention | 3.88       | 1.32     | 80.0% | 100%  |
| DGCNN              | Point Cloud   | CNN+EdgeConv                 | 4.84       | 1.52     | 53.3% | 100%  |
| PointNet++         | Point Cloud   | Hierarchical PointNet        | 8.97       | 2.76     | 6.7%  | 60.0% |
| PointNet           | Point Cloud   | Shared MLP + Max pooling     | 9.49       | 1.88     | 0.0%  | 60.0% |

*Evaluated on 15 test samples from AMBF simulation (150 total dataset)*

# Technical Approach: More to be done!

- 1) Collecting real data
  - a) Setup ros
  - b) stereo calibration
- 2) Validate sim-to-real
  - a) finetune model
  - b) Intrinsic conditioning because simulation is static
- 3) Shadow-Based Depth
- 4) Realtime optimization
- 5) Failure Case Handling
- 6) ...

# Deliverables

## Minimum (Must Have)

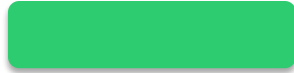
- Working simulation model
- ROS integration for synchronized data capture
- Stereo calibration
- Real data collection
- Fine-tuned model achieving error on real robot
- Real-time inference
- Camera intrinsics generalization
- Documented sim-to-real performance gap analysis

## Stretch (Nice to Have)

- ☐ Sub-millimeter accuracy on real robot
- ☐ Sensor fusion (vision + joint encoders)
- ☐ ICRA 2027 paper submission

# Plan & Management: Timeline Meeting on Mon & Wed (I also micro-update my progress to Prof Adnan in teams)

## Phase 1: Foundation



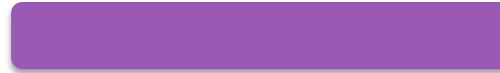
- AMBF setup
- Data pipeline
- Baseline

## Phase 2: Model Dev



- Implement 7 models
- Training
- Evaluation

## Phase 3: Real Data



- ROS setup
- Data collection
- Annotation

## Phase 4: Transfer



- Domain adaptation
- Fine-tuning
- Validation

## Phase 5: Deploy



- ROS integration
- Real-time demo
- Documentation

W1 W2 W3 W4 W5 W6 W7 W8 W9 W10 W11 W12 W13 W14 W15

## References

- [1] He et al., "Deep Residual Learning for Image Recognition", CVPR 2016 (ResNet)
- [2] Qi et al., "PointNet: Deep Learning on Point Sets", CVPR 2017
- [3] Sun et al., "Deep High-Resolution Representation Learning", CVPR 2019 (HRNet)
- [4] Munawar et al., "A Real-Time Dynamic Simulator and an Associated Front-End", IROS 2019 (AMBF)
- [5] i2ris Robot: Integrated Intraocular Robotic System, JHU LCSR
- [6] Wang et al., "Dynamic Graph CNN for Learning on Point Clouds", TOG 2019 (DGCNN)
- [7] Vaswani et al., "Attention Is All You Need", NeurIPS 2017 (Transformer)
- [8] Luvizon et al., "Human Pose Regression with Soft-Argmax", BMVC 2017
- [9] Dosovitskiy et al., "An Image is Worth 16x16 Words", ICLR 2021 (ViT)
- [10] Qi et al., "PointNet++: Deep Hierarchical Feature Learning", NeurIPS 2017

Tools: PyTorch, OpenCV, ROS, NumPy, Matplotlib